

Begonnen am	Donnerstag, 23. Januar 2025, 10:18
Status	Beendet
Beendet am	Freitag, 24. Januar 2025, 11:35
Verbrauchte Zeit	1 Tag 1 Stunde

Information

Hinweise zu den Übungen

Mit den Übungen können Sie das erlernte Wissen und Können festigen.

Gegebenenfalls gibt es "Advanced" Fragen, welche zur weiteren Vertiefung für interessierte Studierende dienen. Diese Aufgaben sind nicht Teil des Prüfungsstoffes im engeren Sinn und als "Advanced" gekennzeichnet..

Bei den meisten Fragen können Sie direkt prüfen, ob Ihre Antwort richtig ist. Gewisse Fragen jedoch (z.B. die offenen Fragen) bieten diese Möglichkeit nicht, weil für diese keine automatisierte Prüfung möglich ist.

Nach Abschluss der gesamten Übung bekommen Sie zusätzlich zu Ihren Resultaten bei etlichen Aufgaben allgemeine Hinweise über die Lösung.

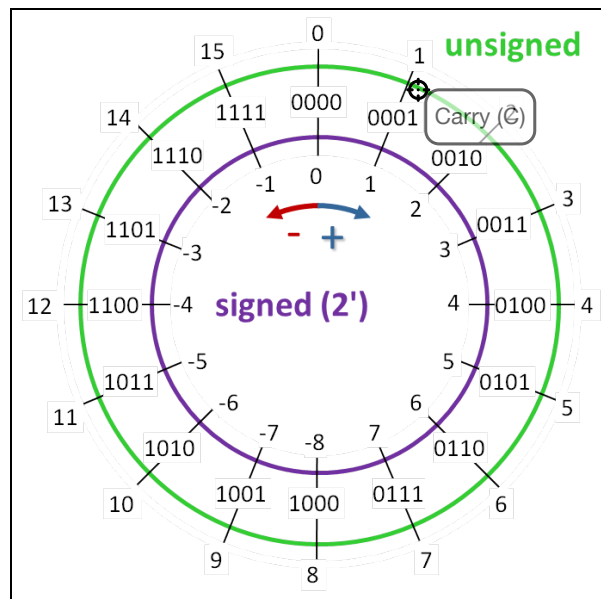
Frage 1

Richtig

Nicht bewertet

Arithmetik und Flags

Wo befinden sich Carry und Overflow? Markieren Sie Die beiden Flags auf dem entsprechenden Zahlenkreis!



Die Antwort ist richtig.

Carry: Dieses Flag wird verwendet um anzuzeigen, dass das Resultat einer **vorzeichenlosen** Operation nicht darstellbar ist. Der richtige Ort ist auf dem **grünen** Kreis zwischen den Zahlen 15 (1111b) und 0 (0000b).

Overflow: Das Flag kennzeichnet bei

vorzeichenbehafteten Operationen den Übergang zwischen der grössten positiven Zahl (7 = 0111b) und der kleinsten negativen Zahl (-8 = 1000b). Der richtige Ort ist auf dem **violetten** Kreis zwischen den Zahlen -8 und 7.

flow (V)

Frage 2

Richtig

Nicht bewertet

Subtraktion einer 4-Bit CPU

Eine CPU besitzt kein eigenes Rechenwerk für Subtraktionen. Wenn die Subtraktion

A - B

ausgeführt werden soll, führt die CPU der Reihe nach die folgenden Operationen durch:

1. Schritt: B wird bitweise invertiert

2. Schritt: A und B werden addiert

3. Schritt:

Zum Resultat wird 1 addiert

Zeigen Sie die Schritte anhand der beiden 4-Bit Zahlen A = 4 und B = 3; das Resultat sollte $4 - 3 = 1$ sein.

Die Antwort ist richtig.

A = 4 = 0100b

B = 3 = 0011b

1. inv(0011b) -> 1100b

2. add(0100b) -> (1)0000b

3. add(0001b) -> 0001b

Die richtige Antwort lautet:

Subtraktion einer 4-Bit CPU

Eine CPU besitzt kein eigenes Rechenwerk für Subtraktionen. Wenn die Subtraktion

A - B

ausgeführt werden soll, führt die CPU der Reihe nach die folgenden Operationen durch:

1. Schritt: [B wird bitweise invertiert]

2. Schritt: [A und B werden addiert]

3. Schritt: [Zum Resultat wird 1 addiert]

Zeigen Sie die Schritte anhand der beiden 4-Bit Zahlen A = 4 und B = 3; das Resultat sollte $4 - 3 = 1$ sein.

Frage 3

Richtig

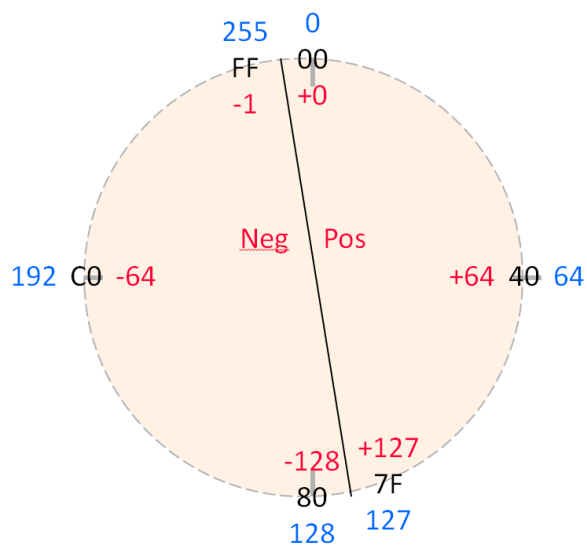
Nicht bewertet

Addition mit einer 8-bit ALU

Eine 8-bit breite ALU addiert jeweils die Operanden op1 + op2. Füllen Sie die Tabelle aus.

op1	op2	Resultat (hex)	carry	overflow
0x82	0x12	0x94	clear	clear
0x34	0x72	0xA6	clear	set
0xC2	0x87	0x49	set	set
0xA3	0x62	0x05	set	clear

Überprüfen Sie die Plausibilität Ihrer Resultate mit untenstehendem Zahlenkreis.



$0x82 + 0x12 = 0x94$, carry clear, overflow clear

$0x34 + 0x72 = 0xA6$, carry clear, overflow set (Resultat ist negativ, obwohl beide Operanden positiv sind)

$0xC2 + 0x87 = 0x49$, carry set (volles Resultat wäre $0x149$), overflow set (Resultat positiv, beide Operanden negativ)

$0xA3 + 0x62 = 0x05$, carry set (volles Resultat wäre $0x105$)

Frage 4

Richtig

Nicht bewertet

Arithmetische Funktionen

Eine Variable ist wie folgt definiert:

Zahl	DCD	0; (32 bit)
------	-----	-------------

Codieren Sie die folgenden Ausdruck in Assembler:

Zahl = Zahl + 15

LDR R1,=Zahl

LDR R0,[R1]

ADDS R0,#15

STR R0,[R1]

MOV R1,#Zahl

LDRH R0,#R1

SUBS R0,R1

LDR R1,#Zahl

MOV R0,R1

ADD R1,=15

Die Antwort ist richtig.

Die richtige Antwort lautet:

Arithmetische Funktionen

Eine Variable ist wie folgt definiert:

Zahl	DCD	0; (32 bit)
------	-----	-------------

Codieren Sie die folgenden Ausdruck in Assembler:

Zahl = Zahl + 15

[LDR R1,=Zahl]

[LDR R0,[R1]]

[ADDS R0,#15]

[STR R0,[R1]]

Frage 5

Richtig

Nicht bewertet

Arithmetische Funktionen

Codieren Sie den folgende Ausdrücke in einer Assemblerdirektive:

$$R2 = R3 + R4$$

$$R0 = R0 + R2$$

$$R0 = R3 - R1$$

$$R0 = R5 - 2$$

$$R1 = R1 + 200$$

$$R3 = -R3$$

$R2 = R3 + R4$: **ADDS R2, R3, R4**

$R0 = R0 + R2$: **ADDS R0, R2** oder **ADDS R0, R0, R2** oder **ADD R0, R2** oder **ADD R0, R0, R2**

$R0 = R3 - R1$: **SUBS R0, R3, R1**

$R0 = R5 - 2$: **SUBS R0, R5, \#2**

$R1 = R1 + 200$: **ADDS R1, \#200**

$R3 = -R3$: **RSBS R3, \#0** oder **RSBS R3, R3, \#0**

Frage 6

Richtig

Nicht bewertet

Rechnen mit Multiword-Operanden

Unsere 32-Bit ARM M0 CPU unterstützt arithmetische Operationen bloss für Operanden, die 32-Bit lang sind (=Registerlänge). Dennoch kann auch mit längeren Operanden gerechnet werden, indem die Rechenoperation in mehrere Stücke aufgeteilt wird. Im folgenden Beispiel sind **zwei 64-Bit Operanden A und B** wie folgt in Prozessor-Registern gespeichert:

- Operand A in R1:R0, d.h. die oberen 32 Bits von A in R1, die unteren 32 Bits von A in R0
- Operand B in R3:R2, d.h. die oberen 32 Bits von B in R3, die unteren 32 Bits von B in R2

Der folgende Code berechnet $A = A + B$:

```
; 64 bit value A is stored in R1 (high part) and R0 (lower part)
; 64 bit value B is stored in R3 (high part) and R2 (lower part)
ADDS R0, R0, R2
ADCS R1, R1, R3
```

Was ist der Unterschied von ADDS und ADCS?

ADCS addiert zusätzlich das Carry-Flag

Welche Aussage ist richtig?

Der Übertrag aus den oberen 32 Bits wird hier nicht be

Warum werden zuerst R0 und R2 addiert? Könnte man auch zuerst R1 und R3 addieren?

Wie würde der Code aussehen, wenn statt einer Addition die Subtraktion $A = A - B$ berechnet werden sollte?

Frage (1): ADCS addiert zusätzlich das Carry-Flag

Frage (2): Der Übertrag aus den oberen 32 Bits wird hier nicht beachtet

Antworten zu den freien Fragen (Warum werden zuerst R0 und R2 addiert? Könnte man auch zuerst R1 und R3 addieren?)

--> Die Addition der beiden Low-Words R0 und R2 liefert den Übertrag (das Carry-Bit!), der zur Summe der High-Words addiert werden muss.

--> Ja, das geht; aber dann ist der Code nicht mehr so kompakt. Um den Übertrag der Low-Words zur Summe der High-Words zu addieren, benötigt es dann extra Instruktionen sowie ein Register, das Null enthält:
MOVS R4, #0 ; lösche ein Register, um später das Carry-Bit zu addieren

```
ADDS R1, R3 ; addiere die beiden High-Words
ADDS R0, R2 ; addiere die beiden Low-Words
ADCS R1, R4 ; mache den Übertrag!
```

Eine Subtraktion $A = A - B$ wäre

```
SUBS R0, R2
```

```
SBCS R1, R3
```

Frage 7

Nicht
beantwortet

Nicht bewertet

Arithmetische Funktionen

Eine Variable ist wie folgt definiert:

Zahl	DCD	0; (32 bit)
------	-----	-------------

Codieren Sie die folgenden Ausdruck in Assembler:

Zahl = Zahl + 2

Verwenden Sie R1 für alle Adressen und R0 für alle Zahlen.

LDR R1,=Zahl

LDR R0,[R1]

ADDS R0,#2

STR R0,[R1]

Frage 8

Richtig

Nicht bewertet

Entstehung des Borrow-Bits

Betrachten Sie die vorzeichenlose Zahl **0x50** (=80) und gehen Sie von einer 8-Bit ALU aus.

Denken Sie daran, dass das Borrow-Flag kein eigenständiges Flag ist, sondern mittels des Carry-Bits ausgedrückt wird: Borrow = !Carry (d.h. Borrow = 1, wenn Carry = 0, und umgekehrt).

Nun **subtrahieren** Sie eine 8-Bit Zahl x. Wie würden Sie alle x beschreiben, für die nach der Subtraktion Borrow = 1 ist (d.h. Carry = 0)? Warum kann es wichtig sein, nach der Subtraktion auf Borrow zu testen?

Welches ist das kleinste x, das Borrow = 1 verursacht?

Welches ist das grösste x, das Borrow = 1 verursacht?

Zu den freien Fragen:

- Die Menge aller x zeichnet sich dadurch aus, dass das Resultat von **0x50 - x kleiner als 0** ist. Bei vorzeichenlosen Zahlen ist dies nicht darstellbar und führt dazu, dass von höheren Bitstellen 256 abgezogen werden muss, damit das Resultat stimmt.
- Das Borrow/Carry-Bit ermöglicht es, Subtraktionen von Zahlen korrekt durchzuführen, die länger als 8 Bits sind.

Kleinstes x = 0x51, weil $0x50 - 0x51 = 0x50 + OC(0x51) + 1 = 0x50 + 0xAE + 1 = 0xFF$ mit carry=0, also borrow=1
Grösstes x = 0xFF, weil $0x50 - 0xFF = 0x50 + OC(0xFF) + 1 = 0x50 + 0x01 + 1 = 0x52$ mit carry=0, also borrow=1
OC(x) ist das One's Complement, also x bitweise invertiert.

Frage 9

Richtig

Nicht bewertet

Entstehung des Carry-Bits

Betrachten Sie die vorzeichenlose Zahl **0xC0** (=192) und gehen Sie von einer 8-Bit ALU aus.

Nun **addieren** Sie dazu eine 8-Bit Zahl x. Wie würden Sie alle x beschreiben, für die nach der Addition das Carry-Flag gesetzt ist? Warum kann es wichtig sein, nach der Addition das Carry-Flag zu checken?

Welches ist das kleinste x, das ein gesetztes Carry verursacht?

Welches ist das grösste x, das ein gesetztes Carry verursacht?

Zu den freien Fragen:

- Die Menge aller x zeichnet sich dadurch aus, dass das Resultat von **0xC0 + x grösser als 255 (=0xFF)** ist und damit **nicht in 8 Bits** dargestellt werden kann.
- Das Carry-Flag muss beachtet werden, wenn eine Summe möglicherweise grösser als 255 wird - dann muss der Übertrag (=Carry) in den nächsten 8-Bit-Block ausgeführt werden. Oder umgekehrt: Das Carry-Flag ist unwichtig, wenn Sie ausschliesslich an den untersten 8 Bits des Resultats interessiert sind.

Kleinstes x = 0x40 (0x3F würde kein Carry erzeugen:

0xC0 + 0x3F = 0xFF mit Carry=0)

Grösstes x = 0xFF

Frage 10

Richtig

Nicht bewertet

Entstehung des Overflow-Bits

Betrachten Sie die positive Zahl **0x60** (=96) und gehen Sie von einer 8-Bit ALU aus.

Nun führen Sie eine **signed Addition** aus und addieren eine 8-Bit Zahl x . Wie würden Sie alle x beschreiben, für die nach der Addition das Overflow-Flag gesetzt ist? Wie kann man generell die Bedingung beschreiben, unter der $\text{Overflow} = 1$ wird? Welche Zahlen x erzeugen in obigem Beispiel **ganz sicher kein** Overflow?

Welches ist das kleinste x , das ein gesetztes Overflow verursacht?

Welches ist das grösste x , das ein gesetztes Overflow verursacht?

Zu den freien Fragen:

- Die Menge aller x zeichnet sich dadurch aus, **$x > 0$** sowie **$0x60 + x < 0$** ist.
- Generell wird das Overflow-Flag gesetzt, wenn das Vorzeichen des Resultats 'falsch' ist, d.h. nicht der Erwartung entspricht. Beispiel: $x = 0x60$, das erwartete Resultat ist positiv: $0x60 + 0x60 = 96 + 96 = 192$. Aber das Resultat 192 ist hexadezimal $0xC0$, was als im Signed-Universum -64 (negativ!) ist.
- Bei einer signed Addition ergeben alle x , die das gegenteilige Vorzeichen haben wie der erste Operand, ganz sicher kein Overflow. Im Beispiel: $0x60$ ist positiv, d.h. sämtliche negativen Zahlen -1 bis -128 können ohne Overflow addiert werden.

Kleinstes $x = 0x20$

Grösstes $x = 0x7F$

Frage 11

Richtig

Nicht bewertet

Entstehung des Overflow-Bits

Betrachten Sie die positive Zahl **0x10** (=16) und gehen Sie von einer 8-Bit ALU aus.

Nun führen Sie eine **signed Subtraktion** aus und subtrahieren eine 8-Bit Zahl x . Wie würden Sie alle x beschreiben, für die nach der Subtraktion das Overflow-Flag gesetzt ist? Wie kann man generell die Bedingung beschreiben, unter der $\text{Overflow} = 1$ wird? Welche Zahlen x erzeugen in obigem Beispiel **ganz sicher kein** Overflow?

Welches ist das kleinste x , das ein gesetztes Overflow verursacht?

Welches ist das grösste x , das ein gesetztes Overflow verursacht?

Zu den freien Fragen:

- Die Menge aller x zeichnet sich dadurch aus, **$x < 0$** sowie **$0x10 - x < 0$** ist.
- Generell wird das Overflow-Flag gesetzt, wenn das Vorzeichen des Resultats 'falsch' ist, d.h. nicht der Erwartung entspricht. Beispiel: $x = 0x90$, das erwartete Resultat ist positiv: $0x10 - 0x90 = 16 - (-112) = 128$. Aber das Resultat 128 ist hexadezimal 0x80, was als im Signed-Universum -128 (negativ!) ist.
- Bei einer signed Subtraktion ergeben alle x , die das identische Vorzeichen haben wie der erste Operand, ganz sicher kein Overflow. Im Beispiel: 0x10 ist positiv, d.h. sämtliche positiven Zahlen 0 bis 127 können ohne Overflow subtrahiert werden.

Kleinstes $x = 0x80$

Grösstes $x = 0x90$



Datenschutz |  Supportanfrage

Frage 12

Richtig

Nicht bewertet

Arithmetische Funktionen und Flags

Gegeben ist der folgende Assembler Code:

```
LDR    R0, =0x65000000
LDR    R1, =0x3D000000
ADDS   R1, R0, R1
```

a) Welcher Wert in Hex steht nach der Ausführung der obenstehenden Befehle in R1?

Antworten mit überzähligen Ziffern werden mit 0 Punkten bewertet.

b) Wie sind die folgenden Flags gesetzt?

N= Z= C= V=

$0x65000000 + 0x3D000000 = 0xA2000000$

$N=1, Z=0, C=0, V=1$

Frage 13

Richtig

Nicht bewertet

Binäre Addition (signed)

Führen Sie die folgende binäre Addition durch. Es handelt sich um 4-bit

vorzeichenbehaftete (signed) Zahlen. Bestimmen Sie, ob ein Overflow entsteht.

Operand 1 Operand 2 Überträge Resultat Overflow

Bilden Sie die Summe, indem Sie die Operanden als farbige Pfeile hintereinander auf dem Zahlenkreis eintragen. Stimmt das Ergebnis mit der Rechnung überein?

```
0110
+0111
01100 Übertrag
----
1101
```

